Given a prompt p and output specification ω, MaLiang-Harness plans the appearance, spatial composition, and temporal dynamics, selects an executable representation and compatible backend b, and generates drawing and animation code. The program may incorporate user-provided assets or, when the desired appearance is difficult to construct through code, assets obtained through image generation or search. The backend renders the resulting representation into an image or video that the MLLM evaluates against the task requirements, grounding subsequent code revisions in observed visual discrepancies rather than execution success alone.

Persistent Executable Generation (PEG) State. The harness maintains a persistent executable generation state that provides a common reference for planning, execution, and revision. At revision k, we define:

$$S_k = \big(P_k, A_k, Z_k, C_k, k\big), \tag{1}$$

where P_k denotes the visual program together with its backend identifier, and A_k contains any associated assets. Program and asset contents are retained with content hashes, while ω specifies the output dimensions and format together with the rendering seed, and any applicable video timing parameters. Z_k describes the spatial composition and temporal dynamics through explicit scene attributes or definitions embedded in P_k, without requiring a separately maintained scene representation. The generation context C_k retains p, ω, user-supplied requirements, and the current generation plan. Planning updates may append requirements but cannot remove or weaken existing ones. Changes to inherited requirements in a separate editing task must cite the user's new instruction.

Initial state. The initial state S_0 contains p and ω and may have no executable content, which is refined through subsequent edits. We distinguish updates to this state from the visual rendering:

$$S_{k+1} = \mathcal{E}(S_k, a_k), \qquad I_k(t) = \mathcal{R}_b(S_k, t; \omega), \tag{2}$$

where $\mathcal{E}$ commits an edit a_k after validating its inputs and checking that its source revision is current. Each commit, including restoration of historical content, receives the next unused revision index within the run. The harness retains the preceding snapshot so that subsequent generation can revisit an earlier version rather than overwrite its executable definition.



Figure 4: **Example of code-driven image synthesis.** The "botanical portrait" is rendered from the visual program P_k of a PEG state without image assets. Algorithm 1 summarizes how the program specifies geometry, appearance, and spatial composition through Canvas drawing operations.

Algorithm 1: Generated Canvas program for T2I.

Require: Canvas 1024×576, seed s
$\mathbf{c} \leftarrow (512, 288)$
$R_\ell \leftarrow \mathrm{PRNG}(\mathrm{hash}(s, \ell))$ for each layer ℓ
Fill the canvas with white.
for $i = 0, \ldots, 25$ **do**
    $\theta_i \leftarrow 2\pi i/26 + \epsilon_i, \quad r_i \leftarrow 167 + 18u_i$
    $\mathbf{p}_i \leftarrow \mathbf{c} + r_i(\cos\theta_i, \sin\theta_i)$
    Draw a curved branch at $\mathbf{p}_i$; add leaves and veins.
end for
Draw Bézier paths for hair, face, and shoulders.
Shade the face; add closed eyes and hair strands.
for each rose at a predefined position **do**
    for $k = 0, \ldots, 4$ **do**
        $n_k \leftarrow [8, 7, 6, 5, 4]_k$
        for $j = 0, \ldots, n_k - 1$ **do**
            $\phi_{k,j} \leftarrow 2\pi j/n_k + \delta_{k,j}$
            Draw a rotated Bézier petal at angle $\phi_{k,j}$.
        end for
    end for
end for
Add daisies, small blossoms, and texture.
Generation finished.

Renderable state. For a renderable state, $\mathcal{R}_b$ evaluates the visual program at content time t under ω using the selected backend b. An image corresponds to evaluation at a specified time, whereas a video is obtained by sampling the temporal evolution encoded in the same representation. The index k therefore tracks revisions of the generation state rather than progression through the generated content; an update to the generation plan can advance k without changing the rendered output. Fig. 4